LAB ASSIGNEMENT -20.1

Name : Akhil
Roll.No : 2303A530024
Batch: 46

Prompt

Generate a secure Python login program using regex for input validation (4–12 alphanumeric) and print result with sample output.

Code:

```
import re
username = input("Enter username: ")
password = input("Enter password: ")
pattern = r'^[a-zA-Z0-9]{4,12}$'
if re.match(pattern, username) and re.match(pattern, password):
    print("Login Successful")
else:
    print("Invalid Input")
```

Output:

```
Enter username: user123
Enter password: pass123
Login Successful
```

Task 2 – SQL Injection Prevention

Prompt:

Generate a Python SQLite program using parameterized queries to safely fetch user details and prevent SQL injection with output.

Code:

```
import sqlite3
conn = sqlite3.connect("users.db")
cursor = conn.cursor()
```

```
cursor.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, age INTEGER)")
cursor.execute("INSERT INTO users VALUES (?, ?)", ("akhil", 21))
name = input("Enter username: ")
cursor.execute("SELECT * FROM users WHERE username = ?", (name,))
result = cursor.fetchone()
if result:
    print("User Found:", result)
else:
    print("User Not Found")
conn.commit()
conn.close()
```

Output:

```
Enter username: akhil
User Found: ('akhil', 21)
```

Task 3 – XSS Prevention

Prompt:

Generate a secure HTML feedback form that prevents XSS using safe text display and show output.

Code:

```
<!DOCTYPE html>
<html>
<body>
<h2>Feedback Form</h2>
<input type="text" id="name" placeholder="Enter name">
<button onclick="show()">Submit</button>
<p id="output"></p>
<script>
function show() {
    let name = document.getElementById("name").value;
    document.getElementById("output").innerText = "Hello " + name;
}
</script>
</body>
</html>
```

Output:

Input: <script>alert(1)</script>
Output on page: Hello <script>alert(1)</script> (no execution → safe)

Task 4 – Security Audit

Prompt:

Generate insecure and secure Python login code showing vulnerabilities and fixes using hashing and validation with output.

Code:

Secure Code

```
import hashlib
import re
username = input("Enter username: ")
password = input("Enter password: ")
if not re.match(r'^[a-zA-Z0-9]{4,12}$', username):
    print("Invalid Input")
else:
    hashed = hashlib.sha256(password.encode()).hexdigest()
    if username == "admin" and hashed == hashlib.sha256("1234".encode()).hexdigest():
        print("Login Success")
    else:
        print("Fail")
```

Output:

Enter username: admin
Enter password: 1234
Login Success

Task 5 – Serialization Security

Prompt:

Generate Python code showing insecure pickle usage and secure JSON alternative with explanation and output.

Secure (JSON)

```
import json
data = {"name": "akhil"}
with open("data.json", "w") as f:
    json.dump(data, f)
with open("data.json", "r") as f:
    d = json.load(f)
print(d)
```

Output:

```
{'name': 'akhil'}
```